

Documento Técnico: Llamadas al Sistema (System Calls) en C

Fecha: 24 de mayo de 2024

Tema: Programación de Sistemas Operativos

Lenguaje: C (Entorno POSIX/Unix-Like)

1. ¿Qué es una Llamada al Sistema (System Call)?

Una Llamada al Sistema es una interfaz de programación que permite que un programa en espacio de usuario (*user space*) solicite un servicio al núcleo del sistema operativo (*kernel space*).

Los programas de usuario no tienen permiso para acceder directamente al hardware (disco duro, memoria RAM, dispositivos de red) por razones de seguridad y estabilidad. Cuando un programa necesita realizar una tarea privilegiada (como leer un archivo o crear un proceso), debe "llamar" al sistema operativo para que este lo haga en su nombre.

Características Clave:

1. Cambio de Modo: Al ejecutar una llamada al sistema, el procesador cambia de Modo Usuario a Modo Kernel.
2. Seguridad: El kernel valida los parámetros para asegurar que el programa no realice acciones maliciosas.
3. Abstracción: Oculta la complejidad del hardware.
4. Costo: Son operaciones relativamente "caras" en términos de rendimiento comparadas con las funciones normales, debido al cambio de contexto.

Diferencia entre Función de Biblioteca y Llamada al Sistema

Es crucial no confundirlas:

- Función de Biblioteca (ej. `printf`, `fopen`): Son funciones escritas en C que se ejecutan en el espacio de usuario. A menudo, estas funciones *internamente* utilizan llamadas al sistema.
 - Llamada al Sistema (ej. `write`, `open`): Son la puerta de entrada directa al kernel.
-

2. Principales Llamadas al Sistema en C

A continuación, se listan las llamadas al sistema más comunes disponibles en sistemas tipo Unix (Linux, macOS) a través de la librería estándar de C.

2.1. Gestión de Archivos (File I/O)

Llamada	Cabecera	Descripción
<code>open()</code>	<code><fcntl.h></code> , <code><unistd.h></code>	Abre un archivo y devuelve un descriptor de archivo (int).
<code>close()</code>	<code><unistd.h></code>	Cierra un descriptor de archivo abierto.
<code>read()</code>	<code><unistd.h></code>	Lee datos de un descriptor de archivo a un búfer.
<code>write()</code>	<code><unistd.h></code>	Escribe datos desde un búfer a un descriptor de archivo.
<code>lseek()</code>	<code><unistd.h></code>	Mueve el puntero de lectura/escritura del archivo.

2.2. Control de Procesos

Llamada	Cabecera	Descripción
<code>fork()</code>	<code><unistd.h></code>	Crea un nuevo proceso (hijo) idéntico al padre.
<code>exec()</code>	<code><unistd.h></code>	Reemplaza el proceso actual con un nuevo programa.

<code>wait()</code> / <code>waitpid()</code>	<code><sys/wait.h></code>	El padre espera a que termine un proceso hijo.
<code>exit()</code>	<code><stdlib.h></code>	Termina el proceso actual.
<code>getpid()</code>	<code><unistd.h></code>	Obtiene el ID del proceso actual.

2.3. Gestión de Memoria

Llamada	Cabecera	Descripción
<code>brk()</code> / <code>sbrk()</code>	<code><unistd.h></code>	Cambia la cantidad de espacio asignado para el segmento de datos.
<code>mmap()</code>	<code><sys/mman.h></code>	Mapea archivos o dispositivos en memoria.

3. Ejemplos de Código en C

Nota: Estos ejemplos están diseñados para compilarse en entornos Linux/Unix.

Para compilar: `gcc programa.c -o programa`

Ejemplo 1: Lectura y Escritura de Archivos (`open`, `write`, `close`)

Este ejemplo crea un archivo y escribe una cadena de texto sin usar `stdio.h` (sin `fprintf`), usando directamente llamadas al sistema.

```
#include <unistd.h> // Para write, close
#include <fcntl.h>  // Para open, O_CREAT, O_WRONLY
#include <stdio.h>  // Solo para perror (manejo de errores)
```

```
int main() {
```

```

int fd;
const char *mensaje = "Hola, esto es una llamada al sistema write.\n";

// 1. Abrir (o crear) el archivo
// O_CREAT: Crear si no existe
// O_WRONLY: Solo escritura
// 0644: Permisos (rw-r--r--)
fd = open("archivo_syscall.txt", O_CREAT | O_WRONLY, 0644);

if (fd == -1) {
    perror("Error al abrir el archivo");
    return 1;
}

// 2. Escribir en el archivo
// write(fd, buffer, cantidad_bytes)
if (write(fd, mensaje, sizeof(mensaje)) == -1) {
    perror("Error al escribir");
    close(fd);
    return 1;
}

// 3. Cerrar el archivo
if (close(fd) == -1) {
    perror("Error al cerrar");
    return 1;
}

printf("Archivo creado y escrito exitosamente.\n");
return 0;
}

```

Ejemplo 2: Creación de Procesos (`fork`, `getpid`)

Este ejemplo demuestra cómo un proceso se duplica a sí mismo. Es fundamental entender que después de `fork()`, hay dos procesos ejecutando el mismo código.

```

#include <unistd.h> // Para fork, getpid, sleep
#include <stdio.h>   // Para printf
#include <sys/types.h> // Para pid_t

```

```

int main() {
    pid_t pid;

    printf("Proceso Principal: Inicio.\n");

    // Llamada al sistema fork
    pid = fork();

    if (pid == -1) {
        // Error
        perror("Error en fork");
        return 1;
    }
    else if (pid == 0) {
        // --- BLOQUE DEL PROCESO HIJO ---
        // fork() devuelve 0 al hijo
        printf("Soy el HIJO. Mi PID es: %d y el de mi padre es: %d\n", getpid(), getppid());
    }
    else {
        // --- BLOQUE DEL PROCESO PADRE ---
        // fork() devuelve el PID del hijo al padre
        printf("Soy el PADRE. Mi PID es: %d y cree al hijo con PID: %d\n", getpid(), pid);

        // Esperar un poco para ver el orden de salida
        sleep(2);
    }

    printf("Proceso %d terminando.\n", getpid());
    return 0;
}

```

4. Manejo de Errores en System Calls

Las llamadas al sistema son propensas a fallar (permisos denegados, memoria insuficiente, archivo no encontrado). Siempre se debe verificar el valor de retorno.

- Valor de retorno: Generalmente, `-1` indica error.
- Variable `errno`: Es una variable global que se establece cuando ocurre un error.

- Función `perror()`: Imprime un mensaje de error legible basado en `errno`.

Ejemplo de patrón de seguridad:

```
if (syscall() == -1) {  
    perror("Descripción del error");  
    // Manejar error (salir, reintentar, etc.)  
}
```

5. Conclusión

Las llamadas al sistema son los cimientos de la programación de sistemas en C. Permiten el control real sobre el sistema operativo, ofreciendo poder y flexibilidad a cambio de una mayor complejidad y responsabilidad en el manejo de errores y recursos.

Mientras que para aplicaciones de negocio se recomienda usar funciones de alto nivel (como las de `stdio.h`), para el desarrollo de servidores, shells, drivers o software de sistema, el dominio de las llamadas al sistema es indispensable.

Referencias:

- *Man pages de Linux* (`man 2 open`, `man 2 fork`)
- *Advanced Programming in the Unix Environment* (W. Richard Stevens)